The Cesame Programming Language

Programming Language Reference Manual

Language Guru:	Qian Zhao (qz2512)
System Architect:	Teng Jiang (tj2488)
Tester:	Yunjia Wang (yw4105)

Contents

1 Introduction **4**

2 Lexical Elements **4**
2.1 Identifiers . . . 4
2.2 Keywords . . . 4
2.3 Separators . . . 4
2.4 Operators . . . 4
2.5 White Space . . . 4
2.6 Comments . . . 5

3 Data Types **6**
3.1 Primitive Data Types . . . 6
3.1.1 bool . . . 6
3.1.2 int . . . 6
3.1.3 float . . . 6
3.1.4 char . . . 6
3.2 Strings . . . 6
3.2.1 Declaring Strings . . . 6
3.2.2 Defining Strings . . . 6
3.2.3 String Member Functions . . . 7
3.2.4 Indexing Strings . . . 7
3.2.5 Assigning Strings . . . 7
3.2.6 Deleting Strings . . . 8
3.3 Structures . . . 8
3.3.1 Declaring Structures . . . 8
3.3.2 Defining and Declaring Variables of Structures . . . 8
3.3.3 Deleting Structures . . . 9
3.3.4 Assigning Structures . . . 9
3.4 Arrays . . . 10
3.4.1 Declaring Arrays . . . 10
3.4.2 Defining Arrays . . . 10
3.4.3 Array Member Functions . . . 11
3.4.4 Indexing Arrays . . . 11
3.4.5 Deleting Arrays . . . 12
3.4.6 Assigning Arrays . . . 12

4 Expressions and Operators **13**
4.1 Operands . . . 13
4.2 Unary Operators . . . 13
4.3 Binary Operators . . . 13
4.3.1 Arithmetic Operators . . . 14
4.3.2 Comparison Operators . . . 14
4.3.3 Logical Operators . . . 14
4.3.4 Bitwise Operators . . . 14

4.3.5 Assignment Operators . . . 15
4.4 Ternary Operators . . . 15
4.5 Expressions . . . 16
4.6 Operator Precedence and Associativity . . . 16

5 Statements **18**
5.1 Blocks . . . 18
5.2 The Empty Statements . . . 18
5.3 Labeled Statement . . . 18
5.4 Expression Statements . . . 18
5.5 Declaration statements . . . 19
5.6 Assignment statements . . . 19
5.7 The `if` Statement . . . 19
5.8 The `while` Statement . . . 19
5.9 The `for` Statement . . . 20
5.10 The `goto` Statement . . . 20
5.11 The `break` Statement . . . 20
5.12 The `continue` Statement . . . 20
5.13 The `return` Statement . . . 20

6 Functions **21**
6.1 Function Declaration . . . 21
6.2 Function Definitions . . . 21
6.3 Calling Functions . . . 22
6.4 Assign Functions . . . 22

7 Sample Program **24**

1 Introduction

Cesame is a C-like programming language that features native support for first-class functions. Cesame also supports non-primitive types such as String, Array, and Struct. Cesame abstracts away pointers and stores most data on the heap, except for primitives and references to objects, similar to Java. The compiler is written in OCaml with C libraries and targets LLVM IR.

2 Lexical Elements

2.1 Identifiers

Identifiers are unique names given to various program elements such as variables, functions, or arrays. Here are the naming rules:

- Case sensitive
- Begin with a letter or an underscore character
- Consist of only letters, digits, or underscore
- Cannot be a keyword
- Cannot contain white space

2.2 Keywords

true	false						
new	delete						
void	bool	int	float	char	struct		
if	else	for	while	goto	break	continue	return
String	Array						
Func	function						
printf							

2.3 Separators

A separator is used to separate tokens, including:

`white_space ( ) { } ; , . :`

2.4 Operators

See the section "Expression and Operators".

2.5 White Space

White space in Cesame refers to characters that are used for formatting, such as spaces, tabs, and newline characters. White space is primarily used to enhance code readability and to separate tokens within a statement.

2.6 Comments

Comments are non-executable lines used to document code and ignored by the compiler. There are two types of comments:

- Line comments

```
// line comments
```

- General comments

```
/* General comments */
```

3 Data Types

3.1 Primitive Data Types

3.1.1 bool

`bool` can be either `true` or `false`. The logical operators can be applied to evaluate bool results.

3.1.2 int

`int` represents whole numbers, both positive and negative, without any fractional or decimal part. `int` occupies 4 bytes (32 bits) of memory, so its range is from -2,147,483,648 to 2,147,483,647.

3.1.3 float

`float` is a double-precision floating point number. It uses 8 bytes (64 bits) of memory and ranges from 1.7e-308 to 1.7e+308.

3.1.4 char

A char is a single printable ASCII character. It can be assigned and defined using an ASCII character or an "escape sequence", such as \n between a pair of single quotation marks. The supported escape sequences are the same as those of C.

```
char a = 'a';
char b = 'b';
char n = '\n';
b = a + 1;
b = 65; // b = 'A'
```

3.2 Strings

A string is an ordered sequence of characters. It can be created and deleted. Cesame has some member functions to help manipulate strings.

3.2.1 Declaring Strings

A string can be declared by the keyword "String" followed by a variable name ending with a semicolon. For example:

```
String s;
```

3.2.2 Defining Strings

A string can be defined with an assignment with a variable name on the left and a sequence of ASCII characters or "escaping sequences" between a pair of normal quotation marks ending

with a semicolon. Definition and declaration in one line are supported. The new keyword is also supported by using a ”new” keyword followed by a keyword ”String” followed by a pair of quotation marks between which are the ASCII characters.

```
String s = "Hello Cesame!\n";
String s1;
s1 = "s";
```

3.2.3 String Member Functions

Suppose that there is a defined string with variable name s, the member functions can be accessed by a dot following the variable name and called like a function. The member function is not over-writable by users. Calling the member function of an undefined string will throw a runtime error but will pass the compilation.

Member function	Equivalent Func object	Functionality
s.len	Func<int>	return the number of characters in the string
s.copy	Func<String>	return a deep copy of the string
s.append	Func<String, String>	return a string with the input appended right after s
s.print	Func<String, void>	print s

```
String s = "Hello Cesame!\n";
s.len(); /*14*/
s.append(s); /*"Hello Cesame!\nHello Cesame!\n"*/
```

3.2.4 Indexing Strings

Indexing a string will return the reference of the character in the corresponding position. The index can be done by following the variable name with a pair of square brackets and an integer n indicating the position between the square brackets with $n \in [0, len(s))$ ending with a semicolon. An out-of-bound indexing will throw a runtime error but will still pass the compilation. Note that indexing is not totally equivalent to Func<int,char> because what it returns is the reference, which means using an index can change the original string.

```
s[1] = 'e'; // s = "Bed"
```

3.2.5 Assigning Strings

Assigning a string to another string will do the reference assignment, which means that these two variables will point to the same string. If a new string is needed, the ”copy” member function is provided to make a deep copy.

```
String s1 = s;
String s2 = s.copy();
s1[1] = 'e'; // s = "Bed", s1 = "Bed", s2 = "Bad"
```

3.2.6 Deleting Strings

A string variable needs to be deleted to free the heap space occupied by it once being defined. It can be done by using the ”delete” keyword followed by the variable name ending with a semicolon. Freeing an undefined string will cause a runtime error but will pass the compilation.

```
String s = "Bad";
delete s;
```

3.3 Structures

A structure is an unordered collection of different variables, including primitive types, String, structure and arrays. It can be declared, defined, and instantiated.

3.3.1 Declaring Structures

The structure is declared using the keyword ”struct” followed by the structure name followed by member declarations between a pair of curly brackets ending with a semicolon. The structure name should follow the naming convention of a non-primitive type, which means that the first letter should be capitalized. The following example declares a structure called Student.

```
    struct Student {
        String name;
        int age;
    };
```

3.3.2 Defining and Declaring Variables of Structures

The structure can be defined in two ways. One is using assignment, the left-hand-side is the structure name followed by the variable name and the right-hand-side is the ”new” keyword followed by the structure name followed by a pair of curly brackets ({}) ending with a semicolon. The members in the structure should be declared within the curly brackets. For example, the structure Student can be defined as

```
    Student a = new Student {
        name = "Qian";
        age = -5;
    };
```

To make the structure definition more like that of C, the assignment can also be omitted.

```
Student a {
    name = "Qian";
    age = -5;
};
```

Also, the variables of structures can be declared and but undefined.

```
Student a;
a {name = "Qian", age = -5};
```

If the keyword "new" is used, the members can be undefined in the definition of the structure variable. Otherwise, the members should be defined at the same time. The member can be accessed and defined by dot operation.

```
Student a, b, c;
a = new Student; // Correct
b {}; // Wrong, compilation error
a.name = "Qian";
a.age = -5;
```

3.3.3 Deleting Structures

Using delete followed by the instance name ending with a semicolon will free the heap space of the instance.

```
struct Student {String name; int age;};
Student stu1 {name = "Qian"; age = -5;};
delete stu1;
```

3.3.4 Assigning Structures

Instances of structures support the assignment operator, with the reference of the right-hand side assigned to the right-hand side, meaning they are pointing to the same instance after the assignment. The deep copy function should be implemented manually.

```
struct Student {String name; int age;};
Func<Student, Student, void> deep_copy_stu = new Func (Student a,
    Student b) -> void {
    b.name = a.name.copy();
    b.age = a.age;
};
Student stu1 {name = "who"; age = -1;};
Student stu2 = stu1;
stu2.age = 7; /*stu1.age = 7*/
Student stu3;
```

```
    deep_copy_stu(stu1, stu3);
    stu3.age = 8; /*stu1.age = 7*/
```

3.4 Arrays

Arrays in Cesame, like in many programming languages, are ordered collections of elements of the same type (both primitive and non-primitive). They allow random access to elements using indices and come with built-in methods for manipulating and accessing array elements.

Furthermore, Cesame allows arrays to be nested, meaning you can have arrays of arrays.

3.4.1 Declaring Arrays

An array is declared using the keyword "Array" followed by arrow brackets to specify explicitly the type of contained elements. The following declarations are samples. Each declaration should end with a semicolon.

```
struct Foo {
    int a;
    int b;
};
Array<int> a1;
Array<String> a2;
Array<Foo> a3;
Array<Array<int>> a4;
Array<Func<int,int>> a5;
```

3.4.2 Defining Arrays

An array is defined either explicitly using the "new" keyword followed by the "Array" keyword followed by arrow brackets (<>) indicating the type and followed by square brackets ([ ]) containing the element separated by the colons. If the array is empty, the square brackets can be omitted. Every definition of an array ends with a semicolon. The following definition is an example.

```
Array<int> a1;
Array<int> a2;
a1 = new Array<int> [1, 2, 3];
a2 = new Array<int> [];
a3 = new Array<int>; // Equivalent to a2
```

Note that the length of the array will be inferred exactly as the number of elements in the definition. For example, the example above has a length of 3 because there are three integers in the array. For an empty array, the length is 0.

A syntactic sugar for defining an array is also provided, which is just the square brackets containing the elements separated by columns ending with a semicolon. For example, the

following definition is equivalent to the array definition in the last example.

```
Array<int> a1;
Array<int> a2;
a1 = [1, 2, 3];
a2 = [];
```

The definition and declaration can be combined in one line.

```
Array<int> a1 = [1, 2, 3];
Array<int> a2 = new Array<int> [];
```

3.4.3 Array Member Functions

The member functions are built-in functions that are not over-writable by users and can be called by an array. The function itself is just like a member in a struct. Suppose that we have already defined a struct T and an array of Array<T> whose name is a, all the member functions are shown in the following table.

Member function	Equivalent Func object	Functionality
a.len	Func<int>	return the number of elements in the array
a.copy	Func<Array<T>>	return a deep copy of the array
a.reverse	Func<Array<T>>	return the reverse of the array
a.insert_front	Func<T,void>	in-place insertion before the first element
a.insert_back	Func<T,void>	in-place insertion after the last element
a.insert	Func<int,T,void>	in-place insertion before nth element

Calling these functions is just like calling a normal function, which is passing input in declared order in a pair of brackets right after the function. For example, a.insert_front(m) is equivalent to a.insert(0,m). Note that if the array is declared but not defined, calling member functions will still pass the compilation but running the code will throw a runtime error.

3.4.4 Indexing Arrays

By providing the index, which is an integer between 0 (included) and the length of the array (not included), within a pair of square brackets right after the name of the array, the user can access any element in this array. The index n means $(n+1)th$ element. Any out-of-bound indexing or indexing in an empty array will pass the compilation but lead to a runtime error. For example, the following function computes the sum of an integer array.

```
Func<Array<int>, int> sum = new Func (Array<int> a) -> int
```

```
{
    int res = 0;
    for (int i = 0; i < a.len(); i++)
        res = res + a[i];
    return res;
}
```

3.4.5 Deleting Arrays

Deleting an array will free all the heap space occupied by this variable. It is done by using the "delete" keyword followed by the array variable name ending with a semicolon. For example, if we have already declared an array a, then we can delete it using:

```
delete a;
```

If the array is declared but not defined, delete it will still pass the compilation but will throw a runtime error.

3.4.6 Assigning Arrays

Assigning an array to another array is supported using the "=" operator, with the array being assigned to the left-hand side, but the effect is assigning the reference of the right-hand-side array, which means that the two variable names point to the same array. If a deep copy of the right-hand-side array is desired, the copy member function should be used. Here are the examples of array assignments. Note that if the array is not defined, assigning another array to it will not be a definition but an assignment by reference.

```
Array<int> a = [1,2,3];
Array<int> b = a; /*Allowed, assign the reference of a to b*/
b[2] = 4; /*a = [1,2,4], b = [1,2,4]*/
Array<int> c = a.copy(); /*Deep copy, definition, C = [1,2,4]*/
delete b; /*a,b all freed, c = [1,2,4]*/
```

4 Expressions and Operators

4.1 Operands

Operands are entities upon which operators act within expressions. They can be values, variables, constants, or the results of sub-expressions. For example, in the expression a + b, a and b are operands.

4.2 Unary Operators

Unary operators are operators in programming that operate on a single operand, meaning they work with only one operand or value.

Symbol	Operator	Description	Example
++	Increment	Increases the value of a variable by 1	`int x = 5;` `x++;`
- -	Decrement	Decreases the value of a variable by 1	`int y = 10;` `y--;`
+	Unary plus	Indicates a positive value	`int z = -5;` `int posZ = +z;`
-	Unary minus	Negates the value of an expression	`int a = 8;` `int negA = -a;`
!	Logical NOT	Performs logical negation	`int isTrue = 1;` `int isFalse = !isTrue;`

4.3 Binary Operators

Binary operators are operators in programming that require two operands to perform an operation.

4.3.1 Arithmetic Operators

Perform mathematical calculations on numeric values.

Symbol	Operator	Description	Example
+	Addition	Adds two operands	`int x1 = 5 + 3;`
-	Subtraction	Subtracts second operand from the first	`int x2 = 7 - 2;`
*	Multiplication	Multiplies two operands	`int x3 = 4 * 6;`
/	Division	Divides first operand by the second	`float x4 = 10.0 / 2.0;`
%	Modulus	Returns the remainder of the division	`int x5 = 10 % 3;`

4.3.2 Comparison Operators

Compare two values and determine the relationship between them.

Symbol	Operator	Description
==	Equal to	Checks if two operands are equal
!=	Not equal to	Checks if two operands are not equal
>	Greater than	Checks if first operand is greater than second
<	Less than	Checks if first operand is less than second
>=	Greater than or equal to	Checks if first operand is greater than or equal to second
<=	Less than or equal to	Checks if first operand is less than or equal to second

4.3.3 Logical Operators

Perform logical operations on boolean values.

Symbol	Operator	Description	Example
&&	Logical AND	Returns true if both operands are true	`a && b`
||	Logical OR	Returns true if either operand is true	`x || y`
!	Logical NOT	Inverts the truth value of its operand	`!a`

4.3.4 Bitwise Operators

Perform operations at the bit level.

Symbol	Operator	Description	Example
&	Bitwise AND	Performs bitwise AND on each pair of bits	`a & b`
|	Bitwise OR	Performs bitwise OR on each pair of bits	`x | y`
∧	Bitwise XOR	Performs bitwise XOR on each pair of bits	`p ^ q`
∼	Bitwise NOT	Inverts all the bits	`~a;`
<<	Left shift	Shifts bits to the left	`x << 2`
>>	Right shift	Shifts bits to the right	`y >> 3`

4.3.5 Assignment Operators

Assign values to variables.

Symbol	Operator	Example
=	Assignment	`int x = 5;`
+=	Addition assignment	`x += 3;`
-=	Subtraction assignment	`x -= 2;`
*=	Multiplication assignment	`x *= 4;`
/=	Division assignment	`x /= 2;`
%=	Modulus assignment	`x %= 3;`
«=	Left shift assignment	`x <<= 2;`
»=	Right shift assignment	`x >>= 3;`
&=	Bitwise AND assignment	`x &= y;`
|=	Bitwise OR assignment	`x |= y;`
∧=	Bitwise XOR assignment	`x ^= y;`

4.4 Ternary Operators

```
condition ? expression1 : expression2;
```

The condition is a boolean expression that results in either true or false. If the condition is

- **true**: expression1 is executed
- **false**: expression2 is executed

4.5 Expressions

An expression is a combination of values, variables, operators, and function calls that evaluates to a single value. It can be as simple as a single variable or value, or it can be complex, involving multiple operations.

Expressions may include:

- **Literals**: Constants such as numbers, characters, or strings.
  - `int num = 10;`
  - `char ch = 'A';`
  - `String s = "Hello";`
- **Variables**: Named storage locations that hold values.
  - `int a = 5;`
  - `float b = 3.14;`
  - `char ch = 'B';`
- **Operators**: Symbols representing computations such as addition, subtraction, division, etc.
  - `int sum = 5 + 3;`
  - `int diff = 7 - 2;`
  - `float div = 10.0 / 2.0;`
- **Function Calls**: Invocations of functions that return values.
  - `int minVal = min(10, 20);`
  - `int absValue = abs(-5);`
  - `float sqrtValue = sqrt(25.0);`

4.6 Operator Precedence and Associativity

The operators are grouped according to the rules of precedence when an expression contains multiple operators. The following list shows types of expressions presented in order of highest precedence first. For the operators of the same precedence, its associativity is left to right unless stated otherwise.

1. Array subscription, function calls, and membership accesses
2. Unary operators. When multiple unary operators are consecutive, the later ones are nested in the earlier ones: !-a means !(-a)
3. Multiplication, division, and modular expressions
4. Addition and subtraction expressions

5. Bitwise shifting expressions
6. >=, <=, >, < expressions
7. ==, != expressions
8. Bitwise AND expressions
9. Bitwise XOR expressions
10. Bitwise OR expressions
11. Logical AND expressions
12. Logical OR expressions
13. Ternary expressions
14. Assignment expressions. Multiple assignment statements are evaluated from right to left.

5 Statements

5.1 Blocks

A block statement is a group of possibly empty statements enclosed in curly braces. Blocks serve several important purposes including grouping statements, scoping and variable lifetime, conditional execution, and code isolation.

```
function print_val () -> int {
    // Block begins
    {
        int a = 5;
        printf("Value of a: %d\n", a);
    }
    // Block ends

    return 0;
};
```

5.2 The Empty Statements

An empty statement does not do anything and is merely a semicolon alone. Mostly, an empty statement is used as the body of a loop statement. It is also used to follow a label that would otherwise be the last statement in a block.

```
for (int i = 0; i < 10; i++)
    ;
```

5.3 Labeled Statement

Labels are identifiers followed by a colon, used to mark a position within the code. These labeled positions can be targeted by control flow statements such as ‘goto‘, ‘break‘, and ‘continue‘. A labeled statement is a statement followed by a label.

Here’s the general syntax for a labeled statement:

```
label_name: statement
```

5.4 Expression Statements

An expression statement is a statement that consists of an expression. Expression statements are only useful for their side effects, such as modifying variables, calling functions, or performing input/output operations.

```
// Function call is an expression statement
printf("Hello, world!\n");
```

5.5 Declaration statements

A declaration statement introduces a new identifier (variable or function name) to the program. It specifies the type of the identifier and optionally initializes it with a value.

```
int x; // Declaration of variable x of type int
```

5.6 Assignment statements

An assignment statement assigns a value to an already declared variable.

```
x = 10; // Assignment of the value 10 to the variable x
int y = x + 5; // Declaration and assignment combined
```

5.7 The `if` Statement

The `if` statement is a flow control statement used for executing a block of code conditionally.

- **`if` Statement**

```
if (condition) {
  // code to be executed when the condition is true
}
```

- **`if-else` Statement**

```
if (condition) {
   // code executed when the condition is true
} else {
   // code executed when the condition is false
}
```

- **`if-else-if` Ladder**

```
if (condition1) {
   // code executed when condition1 is true
} else if (condition2) {
   // code executed when condition1 is false and condition2 is true
} else {
   // code executed when both condition1 and condition2 are false
}
```

5.8 The `while` Statement

The `while` statement is a control flow statement that repeatedly executes a block of code as long as a specified condition is true.

```
while (condition) {
    // Code block to be executed repeatedly
}
```

5.9 The `for` Statement

The `for` statement is a versatile control flow statement used for iterative execution. It allows you to execute a block of code repeatedly, with precise control over the iteration process.

```
for (initialization; condition; step) {
    // Code block to be executed repeatedly
}
```

5.10 The `goto` Statement

The `goto` statement is a control flow statement that allows you to transfer program control to a labeled statement within the same function.

```
goto label;
```

5.11 The `break` Statement

the `break` statement is a control flow statement used to terminate the execution of a loop or switch statement prematurely.

```
break;
```

5.12 The `continue` Statement

The `continue` statement is a control flow statement used to skip the current iteration of a loop and proceed to the next iteration.

```
continue;
```

5.13 The `return` Statement

The `return` statement is used to terminate the execution of a function and return a value to the calling code.

```
return <expression>;
```

6 Functions

Functions are considered ”first-class citizens” in Cesame, which means that a function can be assigned to variables, passed as arguments to other functions, and returned from other functions. Just like other non-primitive types in Cesame, we can declare and define a function object. However, Cesame employs a different syntax for function definition compared to C, reflecting its unique design principles.

6.1 Function Declaration

Functions in Cesame accept zero or more parameters as inputs and return zero or one output. A named function is characterized by the function name and the types of its inputs and output. An anonymous function is only characterized by the types of its inputs and output.

The types of inputs and output are specified in the bracket <> followed by ”Func”, and separated with ”,”. There has to be at least one element in the bracket. The last element should be either a type specifying the output type, or the void keyword indicating that we output type. ”void” keywords that are not the last element are ignored.

```
/* f1 takes in 2 integers and returns an integer. */
Func<int, int, int> f1;

/* f2_1 takes in nothing and returns an integer. */
Func<int> f2_1;

/* void ignored, equivalent with f2_1 */
Func<void, int> f2_2;

/* f3 takes in an integer and returns nothing */
Func<int, void> f3;

/* Functions can both be inputs and outputs and also can take in and return
    non-primitive types. */
Func<Func<String, Array<int>>, Func<String>> f4;
```

6.2 Function Definitions

An anonymous function is defined using the following syntax.

```
/* An anonymous function that increments an integer */
(int x)->int { return x + 1 };

/* An anonymous function prints a string but does not return anything */
(String s)->void { s.print(); };

/* An anonymous function that does not take in anything, just returns 42 */
(String s)->int { return 42; };
```

There are 3 ways to define a function:

Firstly, by using the ”new” keyword we provide just like what we do with another non-primitive type. Notice that because the type is going to be able to be inferred from the definition, we can just use ”new Func” instead of ”new Func<...>”, because the type is specified later in the body of the definition. This is the first kind of type inference allowed in functions.

```
Func<int, int> inc;
inc = new Func (int i)-> int { return i + 1; };
```

If you declare and define a function together, the input and output types can also be inferred.

```
Func inc = new Func (int i)-> int { return i + 1; };
```

Secondly, by using the ”function” keyword (a syntactic sugar) we provide, you can declare and define a function together, similar to the syntax in C.

```
function inc (int i)-> int { return i + 1; };
Func<int, int> inc (int i)-> int { return i + 1; };
```

Thirdly, you can define a function with a defined function, using ”=”, with LHS being the undefined function and RHS being a defined function. Notice that here ”=” is definition, not assignment.

```
/* Function declared, but not defined */
Func<int, int> inc, dec;

function another_dec (int i)-> int { return i - 1; };

/* inc is defined with a lambda function */
inc = (int i)->int { return i + 1 };

/* dec is defined with a named function another_dec */
dec = another_dec;
```

6.3 Calling Functions

Calling a function or in the sense of first-class functions evaluating a function, has a similar syntax with C.

```
Func inc = new Func (int i)->int { return i + 1; };
int a = inc(a);
```

6.4 Assign Functions

Lastly, a defined function (named or anonymous) can be assigned to a defined named function.

```
Func inc = new Func (int i)->int { return i + 1; };
Func another_inc = new Func (int i)->int { return i + 1; };

/* assign a named function to inc */
inc = another_inc;

/* assign a lambda function to inc */
inc = (int i)->int { return i + 1; };
```

7 Sample Program

Here is a sample program in Cesame that does bubble-sort on an array of user-defined struct MyObject. The function `compareMyObject` is passed into `bubbleSort` as a parameter.

```
struct MyObject {
    int index;
    String text;
};

function compareMyObject(MyObject a, MyObject b)->bool {
    return a.index < b.index;
};

function bubbleSort(Array<MyObject> arr, int n,
    func compare(MyObject, MyObject)->bool)->Array<MyObject>
{
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (compare(arr[j], arr[j + 1])) {
                MyObject temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    return arr;
};

Array<int> indices = [1, 3, 2];
Array<int> data = [4, 5, 6];

int n = 3;
Array<MyObject> objects = new(Array<MyObject>);

for (int i = 0; i < n; i++) {
    objects.insert_back(new(MyObject));
    objects[i].index = indices[i];
    objects[i].data = data[i];
}

bubbleSort(objects, n, compareMyObject);

for (int i = 0; i < n; i++) {
    delete objects[i];
}

delete objects;
```